

A Novel Imbalanced Ensemble Learning in Software Defect Predication

JIANMING ZHENG¹, XINGQI WANG¹, DAN WEI¹, BIN CHEN¹, AND YANLI SHAO.¹

¹Key Laboratory of Complex Systems Modeling and Simulation, School of Computer Science, Hangzhou Dianzi University, Xiasha Higher Education Zone, Hangzhou, Zhejiang 310018, P. R. China

Corresponding author: Dan Wei (e-mail: weimd@hdu.edu.cn).

This research work was supported by Zhejiang Provincial Natural Science Foundation of China (Nos. LY20F020015, LY21F020015 and LY21F020017), National Science Foundation of China (Nos. 61702517, 61972121 and 61702146) and China Scholarship Council (CSC) Scholarship.

ABSTRACT With the availability of high-speed Internet and the advent of Internet of Things devices, modern software systems are growing in both size and complexity. Software defect prediction (SDP) guarantees the high quality of such complex systems. However, the characteristics of imbalanced distribution of defect data sets have led to the deviation and loss of accuracy of most software defect prediction methods. This paper presents two novel approaches for learning from imbalanced data sets to produce a higher predictive accuracy over the minority class. These two methods differ in whether the oversampling and the misclassification cost information are utilized during training stage, and they are good at different aspects of imbalanced classification, one for dealing with highly imbalanced data sets and the other for moderately imbalanced data sets. Comparing with other state-of-the-art imbalance learning algorithms on imbalanced datasets, the experimental results show that these two methods have achieved excellent results in terms of G-mean and AUC measures, and more accurately identified the defective modules to reduce the cost of detection system.

INDEX TERMS Class imbalance, cost-sensitive learning, ensemble learning, oversampling, software defect predication.

I. INTRODUCTION

WITH the development of automation and the upgrading of computers, the scale and complexity of software systems are gradually increasing, such as air traffic control systems, autonomous cars, SCADA systems, personal applications, and enterprise applications. Therefore, for software companies, providing safe and reliable software has become an inevitable requirement. However, the potential defects in software pose a serious threat to the quality of software. Research suggests that effective defect prediction could help to locate bugs and allocate testing resources more efficiently, thus it has become an extremely popular research topic [1], [2].

In order to locate and fix software defects as much as possible before the software is released, software defect prediction came into being. Unfortunately, reducing the high debugging costs and the number of software defects is a challenging problem, especially considering the limited testing resources of software team [3], [4], and often facing strong pressure for rapid delivery [3], [5]. Therefore, researchers have introduced machine learning methods to predict defects

in software source code [6], such as Naive Bayes (NB) [7], [8], support vector machine (SVM) [7], decision trees [8], and neural networks [9]. Malhotra *et al.* have been proposed for software defect detection based on the measure of internal metrics and defect data from similar projects or earlier releases to construct defect detection models [2]. These machine learning methods in software defect prediction may help to distribute testing and inspection work more easily and wisely, focusing more attention on software modules that may contain defects.

In addition, there is the characteristic of imbalance distribution in software defect dataset, that is, in a given data set, the representativeness of one type of data is extremely low (minority) relative to the other (majority) [10]. In defect prediction, the number of instances representing the "defective" modules is much less than the number of instances representing the "non-defective" modules. However, the class consist of defective modules is the most important class, it is the class that we want to learn and to predict correctly. Due to the small scale of defective class, traditional machine learning methods without considering the imbalance distribution will give less

weight to instances from defective class, and these methods are often difficult to learn accurately and reliably [11].

At present, the different strategies for solving the class imbalance problem include: 1) Remove some instances from the majority class, or synthesize minority instances. This method solves problem from the data level to balance the number of majority instances and minority instances [12]; 2) Set a higher misclassification cost for minority class, that is, the FN (false negatives) has a higher misclassification cost than the FP (false positives) [12]; 3) Combine sampling method or cost-sensitive method into ensemble learning to improve classification accuracy with better generalization performance of ensemble learning.

Among the above imbalanced learning strategies, the third strategy has better comprehensive performance because it has advantages of both the other two strategies and ensemble learning. Tao *et al.* proposed a self-adaptive cost weights-based support vector machine cost-sensitive ensemble for imbalanced dataset classifications [13]. This algorithm not only applies cost-sensitive SVM as a basic weak learner but also simultaneously modifies the standard AdaBoost scheme to a cost-sensitive one to guarantee the consistency of their optimization objectives. In addition, Li *et al.* proposed a two-stage ensemble learning to deal with class imbalance problem by applying RESample with replacement (RES) technique to train multiple different EMKCA predictors [14]. However, these approaches do not provide a more fine grained classification of the class imbalance problem. The highly imbalanced and slightly imbalanced problems should be resolved separately, because different imbalanced scenarios will have different effects on the performance of the classifier.

In this paper, we propose two new methods to solve class imbalance in software defect prediction. The first one is IAdaBoost (imbalance AdaBoost) algorithm, which improves the AdaBoost algorithm and specializes in dealing with highly imbalanced data sets. The other is SWIMBoost algorithm, based on a combination of the SWIM (Sampling With the Majority) [15] algorithm and the boosting procedure that specializes in dealing with slightly imbalanced data sets. These two methods can minimize the influence of imbalanced distribution in training data to reduce the false negative rate and to improve classification performance.

This paper makes the following contributions:

- 1) We propose IAdaBoost algorithm, which introduces adaptive cost matrix to AdaBoost, making it more suitable for the classification of highly imbalanced data sets.
- 2) We also propose SWIMBoost algorithm, which introduces oversampling method SWIM to synthesize minority samples, making it more suitable for the classification of slightly imbalanced data sets.
- 3) We compare the proposed algorithms with the most classic imbalanced learning algorithms on software defect data sets and demonstrate the superior performance of the proposed algorithms.

The remainder of this paper is organized as follows. Section 2 provides a brief introduction to imbalanced learning methods and summarizes how these methods have been applied in software defect prediction. Section 3 introduces the realization principle of the two algorithms we proposed in detail. Section 4 sets out the data used, the measures used, and the details of our experimental design. In addition, we also have discussed our experimental results. Section 5 presents the threats to external validity and construct validity. Next, Section 6 draws our study conclusions, then arranges the following work.

II. RELATED WORK

The accuracy of software defect prediction models is often hindered by the imbalanced nature of data sets [16], [17]. The main problem is that the commonly used prediction algorithms assume that classes in any data set are balanced [18], [19]. Therefore, models trained on imbalanced software defect data sets tend to favor non-defective instances while ignoring defective instances [1]. As a result, class imbalance has been recognized as one of the main reasons for the poor performance of software defect prediction models [20]. Hence, in the past two decades, many researchers have tried to solve this problem [12], [21], [22]. The existing learning methods used to solve the imbalance problem can be divided into the following three categories:

- Sampling
- Cost-Sensitive Learning
- Ensemble Learning

Sampling is a data-level strategy that uses some mechanism to provide a balanced distribution of imbalanced data sets. Studies show that, compared with imbalanced data sets, balanced data sets can improve the overall classification performance of multiple basic classifiers [23]. These results prove that the application of sampling method in imbalanced learning is effective. Within sampling there are four main approaches. 1) Random Oversampling. It will randomly select a group of minority instances and balance the data distribution by copying selected instances and adding them to the original dataset. This method is simple and feasible, however, it is easy to cause overfitting. 2) Random Undersampling. Unlike random oversampling, which appends data to the original dataset, random undersampling will randomly select a set of majority instances and remove these instances from the original dataset to adjust the balance of the original dataset. Yet this method is prone to cause a loss of dataset information. 3) Synthetic Sampling. In order to avoid the overfitting caused by random oversampling, the synthetic oversampling method balances the distribution of dataset by synthesizing minority instances, in which SMOTE (Synthetic Minority Over-sampling Technique) method has made great success in software defect prediction [10]. Nevertheless SMOTE overlooks the majority class in increasing the minority instances, leaving the increased instances at the decision boundaries or overlaying. Aiming at this problem, Sharma *et al.* proposed a new sampling method [15], which increased

minority instances according to the Mahalanobis distance between majority class and minority class, and achieved good classification results. 4) Cluster-Based Sampling. Most of the previous algorithms only focus on within-class imbalance and ignore between-class imbalance. Jo *et al.* proposed a cluster-based oversampling algorithm CBO [24]. This paper reveals the essence of imbalanced learning problem, that is, the effective way to solve the imbalance problem is to solve the within-class imbalance and the between-class imbalance simultaneously.

Unlike sampling methods attempt to balance distributions by considering the representative proportions of class instances in the distribution, cost-sensitive learning methods consider the costs associated with misclassifying instances [25]. Recent research indicates that there is a strong connection between cost-sensitive learning and learning from imbalanced data sets, therefore, the theoretical foundations and algorithms of cost-sensitive methods can be naturally applied to class imbalance problem [26]. Moreover, various empirical studies have shown that cost-sensitive learning performs better than sampling methods [27] in some application domains, including software defect prediction. Hence, a number of cost-sensitive learning methods have been developed by using cost matrices, such as cost-sensitive K-nearest neighbors [28], cost-sensitive decision trees [29], cost-sensitive neural networks [30], and cost-sensitive support vector machines [31]. Unfortunately, the cost of misclassification is generally determined by the experience of domain experts and rarely by algorithmic learning. However, experts with different experience may give different misclassification cost, which makes the results subjective and unconvincing. In this paper, an automatic learning strategy of misclassification cost is proposed, which attempts to calculate the suitable misclassification cost for each dataset and eliminate human factors.

In recent years, ensemble of classifiers have arisen as a possible solution to the class imbalance problem attracting great interest among researchers. The basic idea is that each classifier is known to make mistakes, but different classifiers have been trained on different data, so the corresponding misclassified instances are not necessarily the same [32]. The most widely used methods in SDP are Bagging [33] and Boosting [34], which combine with sampling technology or cost-sensitive technology to produce a lot of variant algorithms to solve the class imbalance problem. There are four different imbalanced ensemble learning methods: 1) Cost-sensitive Boosting. AdaBoost uses the whole dataset to train each classifier serially, but after each iteration, it gives more focus to misclassified instances, the quantity of focus is measured by a weight, which initially is equal for all instances [35]. In imbalanced datasets, AdaBoost tends to give more weight to majority class, because the majority class contributed a lot to the overall accuracy. For this reason, there are different methods to modify the weight update of AdaBoost, while the cost-sensitive method retains the general learning framework of AdaBoost and introduces the cost matrix into the weight update formula. Many approaches have been pro-

posed to improve the performance of AdaBoost [36]–[38]. Li *et al.* proposed an ensemble algorithm, PSOPD-AdaBoost-A, which can re-initialize parameters to avoid falling into local optimum, and optimize the coefficients of AdaBoost weak classifiers [39]. 2) Boosting-based Ensembles. Embed the sampling in Boosting algorithm, which can change the weight distribution of classifier and make it favor minority class in each iteration. SMOTEBoost [40] and RUSBoost [41] are the most representative algorithms in this family. 3) Bagging-based Ensembles. The hybridization of bagging and data preprocessing techniques is usually simpler than their integration in boosting. Because Bagging algorithms do not need to recalculate any type of weight, their combination is simpler. In this methods, the key factor is how to deal with class imbalance problem to obtain a useful classifier in each iteration without forgetting the importance of diversity. Deng *et al.* combined a cost-sensitive algorithm based on Bagging with the dynamic sampling algorithm, and proposed a multi-fusion imbalanced ensemble learning algorithm [42]. This algorithm has low complexity and can dynamically adjust the cluster according to the iteration of AdaBoost, adaptively synchronized with changes caused by sample weight changes. 4) Hybrid Ensembles. Liu *et al.* proposed two hybrid ensemble algorithms, namely EasyEnsemble and BalanceCascade. The main difference of the algorithms in this category with respect to the previous ones is that they carry out a double ensemble learning, that is, they combine both bagging and boosting, which is called exploratory undersampling technology [43].

To sum up, sampling, cost-sensitive learning, and ensemble learning are commonly used in the software defect prediction to solve imbalance distribution. All approaches have their own advantages in SDP according to the above review to address the class imbalance problem. This motivate us to combine the advantages of each method to achieve more accurate predictive performance. In addition, our research found that there are some flaws in these methods to solve the practical problems, such as how to determine the misclassification cost of minority class in the cost matrix, how to modify the weight updating rule of AdaBoost algorithm for imbalanced problem, and how to combine the various methods. Therefore, our research focus is to improve the commonly used methods to solve the class imbalance problem. On this basis, two novel algorithms are proposed, which are more suitable to deal with the imbalanced problem in software defect prediction.

III. METHODS

In this section, we analyze the defects of AdaBoost algorithm in imbalanced classification, and then improve AdaBoost by combining ensemble learning with the cost-sensitive technology, thus putting forward IAdaBoost algorithm, which has better performance for highly imbalanced datasets. Detailed steps are described in Algorithm 2. Moreover, the performance of IAdaBoost is not obviously improved for slightly imbalanced datasets, so we further analyze this situation and

introduce SWIM resampling technology. On this basis, we propose the SWIMBoost algorithm, which achieves better classification performance for slightly imbalanced datasets. Detailed steps are described in Algorithm 3. Finally, we introduce the whole working process of defect prediction system.

A. DEFECTS OF ADABOOST ALGORITHM IN IMBALANCED CLASSIFICATION.

As previously reported in [35], given a training set $\{(x_1, y_1), \dots, (x_n, y_n)\}$, where x_i is an instance with m-tuple of attribute values which belongs to a certain instance space X and $y_i \in Y = \{-1, 1\}$ is a label. Specifically, for a binary-class imbalanced dataset, if x_i belongs to the majority class, $y_i = -1$, and otherwise $y_i = 1$. The pseudocode for AdaBoost is shown in Algorithm 1.

Algorithm 1 AdaBoost

Input:

Training data set $T = \{(x_1, y_1), \dots, (x_n, y_n)\}$.

Output:

The final hypothesis $H(x)$.

- 1: Initialize $W^1(i) = \frac{1}{n}$.
- 2: **for** each $t \in [1, K]$ **do**
- 3: (a) Train weak learner h_t using distribution W^t .
- 4: (b) Compute the classification error rate of h_t on the T ,

$$err_t = \sum_{i=1}^n W^t(i) I(h_t(x_i) \neq y_i).$$

- 5: (c) Compute the coefficient α_t of h_t ,

$$\alpha_t = \frac{1}{2} * \ln \left(\frac{1 - err_t}{err_t} \right).$$

- 6: (d) Update and normalize instance weights W^{t+1} ,

$$W^{t+1}(i) = \frac{W^t(i)}{Z_t} * \exp(-\alpha_t h_t(x_i) y_i)$$

where Z_t is a normalization factor.

- 7: **end for**

- 8: **return** For a test instance x ,

$$H(x) = \text{sign} \left(\sum_{m=1}^K \alpha_m h_m(x) \right).$$

It has been shown in [13] that the solution regarding h_t in step 6 is obtained as follows:

$$h_t = \arg W^t(i) I(h_t(x_i) \neq y_i) \quad (1)$$

where $I(\pi) = 1$ if π hold 1, and otherwise $I(\pi) = 0$. Therefore, the optimal classifier h_t should be one that minimizes the weighted error rate based on the current data distribution.

As described above, the main idea of AdaBoost weighting strategy is to increase weights of misclassified instances

and decrease weights of correctly classified samples. However, this weighting strategy is not suitable for imbalanced classification problems. For imbalanced classification problems, a favorable weighting strategy should be one that is able to distinguish different classes of instances, and boost more weights on those instances associated with higher identification importance, because the correct classification of instances in the minority class is more important than the contrary case. Unlike traditional AdaBoost algorithm, which increases (or decreases) the misclassified (or correctly classified) instances from different classes by an identical ratio, our proposed approach can boost more weights on minority instances associated with higher identification importance, which is more applicable for imbalanced dataset classifications.

B. IADABOOST ALGORITHM

We introduce the cost-sensitive learning to AdaBoost algorithm by using cost matrix to improve the prediction accuracy of minority class. Meanwhile, ensemble learning is used to ensure the prediction accuracy of majority class as far as possible, so as to improve the overall prediction performance of imbalanced data sets. There are three main difference between IAdaBoost and AdaBoost: 1) Initialization. In AdaBoost, all instances are given the same initial weight. In contrast, IAdaBoost uses equation (2) to calculate the initial weight of each instance, so that the weights of minority instances are higher than that of majority instances. In this way, classifier will pay more attention to the minority instances. 2) Re-weighting. When instances weights are updated at the end of each iteration, AdaBoost increases the misclassified instances from different classes by an identical ratio. On the contrary, misclassified instances from minority class are assigned higher weights in IAdaBoost algorithm. Equation (3) defines the process of re-weighting. 3) Voting. When model is built to predict the test set, AdaBoost is a weighted vote based on the prediction results of each weak classifier, while IAdaBoost is determined based on the total cost of each weak classifier when the current instance is predicted into different class, and then the class with low total cost is selected as the final prediction result. Algorithm 2 presents the complete process, in which adaptive cost matrix strategy is described in steps 1 to 8, initialization is described in step 9, the re-weighting is described in steps 10 to 15, and concludes with step 16 for the Voting. Each component or phase is described in the following paragraphs.

$$W^t(i) = \frac{C^i}{\sum_{j=1}^n C^j} \quad (2)$$

Given a training set T containing n instances, $W^t(i)$ denotes the weight of the i th instance at the t th iteration, and C^i is the cost of misclassifying the i th instance.

After modifying the initial weight of the training instances according to equation (2), IAdaBoost algorithm will train several independent weak classifiers. At the end of each

$$W^{t+1}(i) = \begin{cases} \frac{W^t(i)}{Z_t} * \exp(\alpha_t * \text{cost}(\text{actual}(i), \text{predicted}(i))), & \text{actual}(i) \neq \text{predicted}(i) \\ \frac{W^t(i)}{Z_t} * \exp(-\alpha_t), & \text{actual}(i) = \text{predicted}(i) \end{cases} \quad (3)$$

iteration, weight of instances will be adjusted to characterize the importance of each training instance in the next iteration. The process of weight adjustment will use cost matrix, which effectively improves weight of the misclassified instances. Moreover, due to different misclassification costs between minority instances and majority instances, the increased weight of minority instances is higher than majority instances. The weight adjustment formula is defined as equation (3).

Where Z_t is normalizing factor, α_t is the weight of the t th weak classifier, $\text{actual}(i)$ is real class of the i th instance in training set, and $\text{predicted}(i)$ is predicted class of the i th instance. $\text{Cost}(i, j)$ means the misclassification cost of predicting the class i instance as the class j instance. $\text{Cost}(i, j)$ can be obtained from the cost matrix.

After K round iteration, K weak classifiers are generated, which are finally combined into a composite classifier. Given an instance, the final classification of composite classifier relies on the votes of all weak classifiers. Since we use the expected misclassification cost to select predicted class, voting is not simply summing up the vote of every weak classifier. Therefore, we propose equation (4) as new weighted voting strategy.

$$H^j(x) = \sum_{m=1}^K \alpha_t * \text{cost}(h_t(x), j) \quad (4)$$

Where $h_t(x)$ is prediction result of the t th weak classifier on the instance x . To classify instance x , $H^j(x)$ is computed for every class. The instance x is assigned to class j with the smallest value for $H^j(x)$. That is, $H^j(x) < H^{j'}(x)$ for all $j' \neq j$.

IAdaBoost algorithm uses cost matrix in three stages including initialization of instances weight, weight update rules of instances, and voting. So the cost matrix is very important to this algorithm. In particular, we propose adaptive cost matrix strategy for calculating the cost matrix value. For binary-class classification, the misclassification costs can be specified in a cost matrix of size 2×2 . The value on the diagonal is cost of the correct classification, and it is defaulted to 0 in this paper. Furthermore, the value on the anti-diagonal is misclassification cost, and the misclassification cost of minority class is greater than that of majority class. In this paper, $\text{cost}(i, j) \geq 1$ if $i \neq j$. The misclassification cost of majority class is set as 1, and misclassification cost of minority class is initialized as 2. Then, for different data sets, IAdaBoost algorithm will automatically select the most appropriate misclassification cost of minority class by using validation set, as shown in Fig. 1.

Algorithm 2 IAdaBoost

Input:

Training data set $T = \{(x_1, y_1), \dots, (x_n, y_n)\}$.
The number of weak classifiers K .

Output:

The final classifier $H^j(x)$.

- 1: Training data set T is divided into training data set T' and validation data set V .
- 2: Calculate the misclassification cost v of minority class by performing steps 3 to 8.
- 3: **for** each $v \in [1, K]$ **do**
- 4: Perform steps 9 to 16 according to current v to train the final classifier $H_v^j(x)$.
- 5: Calculate the $G\text{-mean}_v$ value of the classifier $H_v^j(x)$ on the validation data set V .
- 6: **end for**
- 7: Calculate the maximum $G\text{-mean}$
 $G\text{-mean} = \text{Max}(G\text{-mean}_v) \text{ for } v \in [1, K]$.
- 8: The misclassification cost of the optimal minority class is the v in the maximum $G\text{-mean}$.
- 9: Initialize the distribution W^1 over the instances by using equation (2).
- 10: **for** each $t \in [1, K]$ **do**
- 11: (a) Train weak learner h_t using distribution W^t .
- 12: (b) Compute the classification error rate of h_t on the T ,
$$\text{err}_t = \sum_{i=1}^n W^t(i) I(h_t(x_i) \neq y_i).$$
- 13: (c) Compute the coefficient α_t of h_t ,
$$\alpha_t = \frac{1}{2} * \ln \left(\frac{1 - \text{err}_t}{\text{err}_t} \right).$$
- 14: (d) Update and normalize W^{t+1} as mentioned in equation (3).
- 15: **end for**
- 16: **return** For a test instance x ,

$$H^j(x) = \sum_{m=1}^K \alpha_t * \text{cost}(h_t(x), j).$$

C. SWIMBOOST ALGORITHM

When a series of weak classifiers are generated, IAdaBoost algorithm increases the weight of misclassified minority instances by using cost matrix, so that classifier pays more attention to the misclassified minority instances and thus improves prediction accuracy of the minority class. However, the performance improvement of IAdaBoost algo-

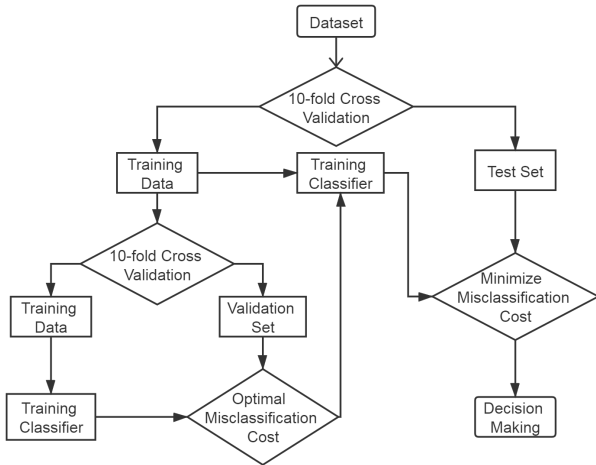


FIGURE 1. Process diagram of IAdaBoost.

algorithm is not obvious for slightly imbalanced data sets, so we propose SWIMBoost algorithm. SWIMBoost algorithm uses SWIM oversampling method to synthesize minority instances, which effectively improves classification accuracy for slightly imbalanced data sets by expanding the decision boundaries of minority class. There are two main difference between SWIMBoost and IAdaBoost: 1) SWIMBoost includes the over-sampling process. 2) SWIMBoost uses the cost matrix only during the testing phase instead of using cost matrix in training and testing. Besides, other steps are the same as IAdaBoost algorithm. Algorithm 3 presents the complete process, in which over-sampling is described in step 1, adaptive cost matrix strategy is described in steps 2 to 9, and step 17 uses cost matrix. Each component or phase is described in the following paragraphs.

The advantage of SWIMBoost algorithm is that it uses the SWIM sampling method to synthesize minority instances, which can enrich the diversity of the minority class for getting more accurate classification. The previous oversampling methods concentrate on the characteristics of minority class and use them to guide the oversampling process. By completely overlooking majority class, they lose a global view on classification problem and, while alleviating the class imbalance, may negatively impact learnability by generating borderline or overlapping instances. Particularly, SWIM oversampling method is to increase minority instances according to the Mahalanobis distance between majority class and minority class. By using the Mahalanobis distance to calculate density distribution of majority class, SWIMBoost make full use of the distribution information of majority class to avoid generating minority instances in high density area. The newly generated minority instances are in the same density area as the original minority instances, so that these minority instances have higher availability and greatly avoid the probability of noise.

Due to SWIMBoost algorithm balances the distribution of

dataset by synthesizing minority instances, we remove the cost matrix during training stage and treat majority instances and minority instances equally to give them same weight.

Algorithm 3 SWIMBoost

Input:

Training data set $T = \{(x_1, y_1), \dots, (x_n, y_n)\}$, where T_+ corresponds to a minority class and T_- corresponds to a majority class.

The number of weak classifiers K .

Output:

The final hypothesis $H^j(x)$.

- 1: Redistribute T_+ with SWIM method $T_{SWIM+} = SWIM(T_+)$, $T = (T_{SWIM+} \cup T_-)$.
- 2: Training data set T is divided into training data set T' and validation data set V .
- 3: Calculate the misclassification cost v of minority class by performing steps 4 to 9.
- 4: **for** each $v \in [1, K]$ **do**
- 5: Perform steps 10 to 17 according to current v to train the final classifier $H_v^j(x)$.
- 6: Calculate the $G\text{-mean}_v$ value of the classifier $H_v^j(x)$ on the validation data set V .
- 7: **end for**
- 8: Calculate the maximum $G\text{-mean}$
 $G\text{-mean} = \text{Max}(G\text{-mean}_v) \text{ for } v \in [1, K]$.
- 9: The misclassification cost of the optimal minority class is the v in the maximum $G\text{-mean}$.
- 10: Initialize the distribution W^1 over the instances by using equation (2).
- 11: **for** each $t \in [1, K]$ **do**
- 12: (a) Train weak learner h_t using distribution W^t .
- 13: (b) Compute the classification error rate of h_t on the T ,

$$err_t = \sum_{i=1}^n W^t(i) I(h_t(x_i) \neq y_i).$$

- 14: (c) Compute the coefficient α_t of h_t ,

$$\alpha_t = \frac{1}{2} * \ln \left(\frac{1 - err_t}{err_t} \right).$$

- 15: (d) Update and normalize W^{t+1} ,

$$W^{t+1}(i) = \frac{W^t(i)}{Z_t} * \exp(-\alpha_t y_i h_t(x_i)).$$

- 16: **end for**

- 17: **return** For a test instance x ,

$$H^j(x) = \sum_{m=1}^K \alpha_t * \text{cost}(h_t(x), j).$$

D. DEFECT PREDICTION SYSTEM

IAdaBoost and SWIMBoost algorithms are the core technologies of defect prediction system. The working process of

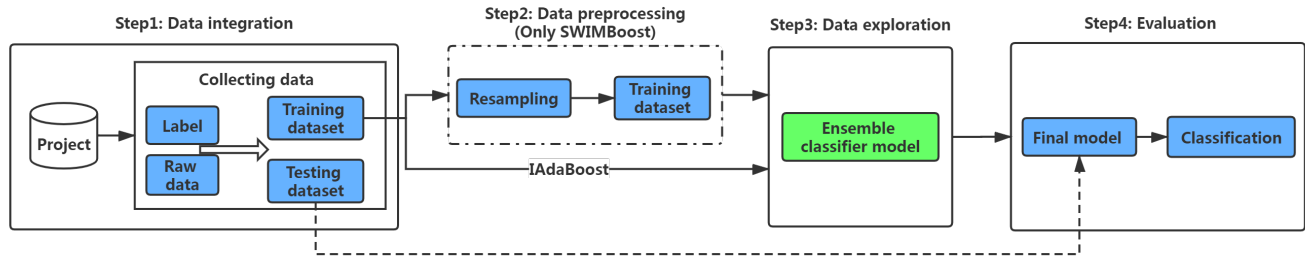


FIGURE 2. Process diagram of defect prediction system.

TABLE 1. Details of the NASA data repository

Dataset	Modules	Minority	Majority	Features	IR
CM1	344	42	302	37	7.19
JM1	9593	1759	7834	21	4.45
KC1	2096	325	1771	21	5.45
KC3	200	36	164	39	4.56
MC1	9277	68	9209	38	135.4
MC2	127	44	83	39	1.89
MW1	264	27	237	37	8.78
PC1	759	61	698	37	11.44
PC3	1125	140	985	37	7.04
PC4	1399	178	1221	37	6.86
PC5	17001	503	16498	38	32.80

the defect prediction system is shown in Fig. 2. The specific steps are as follows:

Step1 : Data integration: Collect data in the project, and then divide them into training dataset and testing dataset.

Step2 : Data preprocessing: Balance the distribution of dataset by synthesizing minority instances.

Step3 : Data exploration: Learn training set to build model by applying IAdaBoost or SWIMBoost algorithm.

Step4 : Evaluation: Classify testing dataset and identify defective instances to help locate bugs.

IV. EXPERIMENTS

In this section, we present details of the experimental study developed. Section A lists details of the data sets used in experiments. Section B explains the evaluation measures used in experiments. Then Section C not only evaluates performance of IAdaBoost and SWIMBoost by comparing them with AdaBoost, RUSBoost, and SMOTEBoost, but also analyzes the significance of method's results through statistical test. Finally Section D discusses how the numbers of selected weak classifiers impact the performance of the proposed algorithm as well as the different classifiers, and relationship between the number of synthesized minority instances and the performance of the SWIMBoost algorithm.

A. DATA SETS

The datasets used in the experiments were collected from the AEEEM [44] and NASA [45] repositories, both software engineering databases. We need to have a consistent granularity

TABLE 2. Details of the AEEEM data repository

Dataset	Modules	Minority	Majority	Features	IR
EQ	324	129	195	61	1.5
JDT	997	206	791	61	3.8
LC	691	64	627	61	9.8
ML	1862	245	1617	61	6.6
PDE	1497	209	1288	61	6.2

and input metrics data. This requires not only the availability of defect data but also the availability of software metrics, so we exclude datasets with a size of less than 120 modules, and finally select a total of 16 data sets from AEEEM and NASA data repository as experimental datasets. The details of these data sets we selected are shown in Table 1 and Table 2.

Precisely, Table 1 and Table 2 are the detailed information of NASA and AEEEM, including the number of total modules, the number of defective modules, the number of non-defective modules, the number of features, and the imbalance rate (IR) of each data set. The imbalance rate describes the imbalance degree of the data set. The greater the imbalance rate, the higher the imbalance degree of the data set. It is assumed that the number of defective modules is recorded as *Minor* and the non-defective module is called *Major*, so

$$IR = \frac{Major}{Minor}. \quad (5)$$

These datasets are varied in their number of instances, features, and imbalance rate, which provide a broad space for evaluating the performance of unbalance learning algorithm.

B. EVALUATION MEASUREMENT

Evaluation measurement plays an important role in evaluating classification performance and guiding classifier modeling. In software defect prediction, the commonly used evaluation measurement for unbalanced data include ROC (the Receiver Operating Characteristic curve), AUC, and several measures based on confusion matrix, such as recall, precision, F-measure, and G-mean [46]. Defective modules with fewer instances are regarded as positive class, marked as 1 in the data set, and non-defective modules with more instances are regarded as negative class, marked as 0. In fact,

TABLE 3. Confusion matrix

	Predict defective	Predict non-defective
Defective	True positive(TP)	False negative(FN)
Non-defective	False positive(FP)	True negative(TN)

we divide the dataset into four categories according to the actual class and predictive class, as shown in Table 3.

As the most commonly used measure in the F-measure family, F1 is the harmonic mean of recall and precision, which is defined as (6). Where Recall is the percentage of positive instances correctly classified, and Precision is the percentage of true positive instances in the instances predicted to be positive.

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} \quad (6)$$

However, F1 is potentially problematic due to excluding True Negatives (TN) in its calculation. F1 originated from the information retrieval domain where typically the number of true negatives, *e.g.*, irrelevant web pages that are correctly not returned is neither knowable nor interesting. Unlike recommendation tasks, this is not so for defect prediction because test managers are definitely interested to know if modules are truly non-defective. Another measurement that covers the entire confusion matrix is G-mean, which is defined as the geometric mean of the two classes of accuracy (see (7)) and is developed specifically for assessing the performance under imbalanced domains, assuming that the precision weights of the two classes are equal.

$$G-mean = \sqrt{TPR * TNR} \quad (7)$$

Where TPR (True positive rate) is the percentage of positive instances correctly classified, and TNR (True negative rate) is the percentage of negative instances correctly classified.

ROC describes the performance evaluation comparison of different classifiers in the machine learning domain. It classifies test instances according to different thresholds and predicts them one by one as a positive class instance. ROC curve is obtained by plotting the results as vertical and horizontal, with false positive rate on the horizontal axis and true positive rate on the vertical axis. AUC is the area of the lower part surrounded by ROC curve and coordinate axis. AUC is also a comprehensive performance measure, so it is widely used as an evaluation measure for defect prediction [47].

In conclusion, G-mean and AUC is used for evaluation measurements in this paper.

C. IADABOOST AND SWIMBOOST COMPARED WITH THE TRADITIONAL IMBALANCED ALGORITHMS.

By combining multiple weak learners, ensemble learning achieves better generalization performance than a single learner. In software defect prediction, ensemble learning is

TABLE 4. Basic information for algorithm parameters in performance comparison (N_estimators corresponds to the maximum number of weak classifiers).

Algorithm	Weak classifier	N_estimators	Learning rate
AdaBoost	CART	50	1
RUSBoost	CART	50	1
SMOTEBoost	CART	50	1
IAdaBoost	CART	50	1
SWIMBoost	CART	50	1

an important method to solve class imbalance. As a typical representative of ensemble learning, AdaBoost has been widely used by researchers. Consequently, our experiment takes AdaBoost algorithm as the baseline model. In addition, SMOTEBoost and RUSBoost algorithms combine resampling method with ensemble learning to achieve superior performance when dealing with class imbalances. Hence, we compare our proposed two methods with three imbalanced learning techniques, that is AdaBoost, RUSBoost, and SMOTEBoost. To ensure fairness of the experiment, all classifiers use the same parameter settings. For instance, CART decision trees are used as base classifiers, the maximum number of weak classifiers is set to 50, and the learning rate is equal to 1. All the reported results are averaged based on 100 independent trials. Overall, the complete algorithm list compared is summarized in Table 4.

The experiment first discusses the main problems we are facing: how our proposed methods compared with existing classic algorithms on different data sets. Table 5 lists the G-mean values of AdaBoost, RUSBoost, SMOTEBoost, IAdaBoost and SWIMBoost. Meanwhile, Table 6 lists the AUC values of AdaBoost, RUSBoost, SMOTEBoost, IAdaBoost and SWIMBoost. The optimal values on each dataset in the table are shown in bold.

According to the experimental results, we can see that the performance of AdaBoost algorithm as the baseline method is the lowest in G-mean and AUC values. From table 5, compared with AdaBoost algorithm, the average G-mean value of RUSBoost algorithm on 16 datasets has been improved by 17%, SMOTEBoost algorithm has been improved by 16%, IAdaBoost algorithm has been improved by 23%, and SWIMBoost algorithm has been improved by 25%. From table 6, compared with AdaBoost algorithm, the average AUC value of RUSBoost algorithm on 16 datasets has been improved by 7%, SMOTEBoost algorithm has been improved by 9%, and SWIMBoost algorithm has been improved by 13%. Among the five algorithms, SWIMBoost has the better comprehensive performance, whose G-mean and AUC values are obviously improved than the other algorithms.

For data sets with high imbalance rate, such as MC1 ($IR = 135.4$), PC5 ($IR = 32.8$), IAdaBoost algorithm achieves the optimal values, where g-mean value is 0.90 and 0.92 respectively, and AUC value is 0.91 and 0.92 respectively. This is because the IAdaBoost algorithm using the idea of cost sensitivity, which can automatically adjust the misclas-

TABLE 5. G-mean measures of AdaBoost, RUSBoost, SMOTEBoost, IAdaBoost and SWIMBoost.

Dataset	AdaBoost	RUSBoost	SMOTEBoost	IAdaBoost	SWIMBoost
CM1	0.28	0.48	0.50	0.69	0.62
JM1	0.22	0.62	0.61	0.62	0.65
KC1	0.38	0.66	0.64	0.69	0.70
KC3	0.52	0.34	0.57	0.57	0.64
MC1	0.32	0.89	0.60	0.90	0.83
MC2	0.61	0.33	0.55	0.41	0.72
MW1	0.35	0.52	0.38	0.60	0.45
PC1	0.33	0.66	0.52	0.75	0.74
PC3	0.10	0.67	0.59	0.74	0.72
PC4	0.61	0.80	0.81	0.85	0.86
PC5	0.53	0.92	0.86	0.92	0.91
EQ	0.74	0.68	0.72	0.71	0.76
JDT	0.71	0.70	0.74	0.72	0.77
LC	0.63	0.65	0.66	0.68	0.70
ML	0.58	0.67	0.68	0.71	0.73
PDE	0.58	0.62	0.63	0.66	0.67
Mean	0.47	0.64	0.63	0.70	0.72

TABLE 6. AUC measures of AdaBoost, RUSBoost, SMOTEBoost, IAdaBoost and SWIMBoost.

Dataset	AdaBoost	RUSBoost	SMOTEBoost	IAdaBoost	SWIMBoost
CM1	0.55	0.60	0.62	0.72	0.71
JM1	0.52	0.62	0.64	0.64	0.65
KC1	0.57	0.68	0.67	0.70	0.71
KC3	0.64	0.56	0.66	0.64	0.68
MC1	0.59	0.89	0.69	0.91	0.85
MC2	0.66	0.53	0.63	0.57	0.67
MW1	0.59	0.59	0.59	0.62	0.66
PC1	0.57	0.68	0.62	0.75	0.77
PC3	0.51	0.71	0.64	0.75	0.73
PC4	0.68	0.81	0.82	0.86	0.87
PC5	0.64	0.92	0.86	0.92	0.92
EQ	0.74	0.68	0.72	0.70	0.76
JDT	0.71	0.70	0.74	0.72	0.77
LC	0.63	0.65	0.66	0.68	0.70
ML	0.58	0.67	0.68	0.71	0.73
PDE	0.58	0.62	0.63	0.66	0.67
Mean	0.61	0.68	0.68	0.70	0.74

sification cost for different data sets. When the imbalance rate is higher, the misclassification cost of minority class will also be higher. The classifier is more inclined to divide the test instances into minority class to solve the problem of skew distribution of instances in terms of the algorithm level. In addition, we can also observe that RUSBoost, SMOTEBoost, and IAdaBoost algorithms achieve lower G-mean and AUC values than AdaBoost algorithm on the slightly imbalanced datasets, such as MC2(IR=1.89) and EQ(IR=1.5). Because there is little difference in the number of instances between the majority class and the minority class, that is, the datasets approximate a balanced distribution. In this case, if the cost sensitive strategy continues to be used, it is easy to increase the FP value in the confusion matrix and thus degrade the performance of the IAdaBoost, so the AdaBoost algorithm can achieve better results.

In order to solve the problem of poor performance of IAdaBoost algorithm when the imbalance rate is low, SWIMBoost algorithm is proposed. On the one hand, SWIM resampling method is used to generate minority instances to expand minority class decision boundary, so that the distribution of instances in training set tends to be balanced, and then the

model is established. On the other hand, in the prediction stage, the cost matrix is used to increase misclassification cost of minority class, which makes the classifier pay more attention to minority instances and improve the prediction accuracy of minority class. According to the experimental results, it can be seen that SWIMBoost achieved 10 optimal g-mean values and 13 optimal AUC values in 16 data sets, and the imbalance rate of these data sets is generally less than 10. Therefore, SWIMBoost algorithm is suitable for dealing with problems with low imbalance rates, and IAdaBoost algorithm is good at issues with high imbalance rates.

In order to compare the performance of different predictive models more clearly, boxplot is used to describe them, as shown in Fig. 3 and Fig. 4. The horizontal axis represents each predictive model, and the vertical axis represents g-mean and AUC values respectively. Each box in the boxplot represents a predictive model that evaluates the distribution of measures on 16 data sets. If the values of a group of measures are more discrete, the box shape is larger, which indicates that the performance of the predictive model is more unstable. Conversely, if the box shape is smaller, it indicates that the performance of the predictive model is

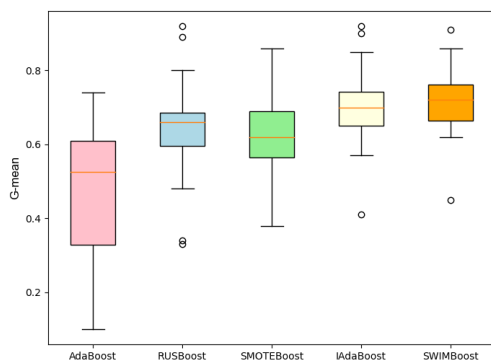


FIGURE 3. Comparison of G-mean measures of predictive models such as AdaBoost, RUSBoost, SMOTEBoost, IAdaBoost and SWIMBoost.

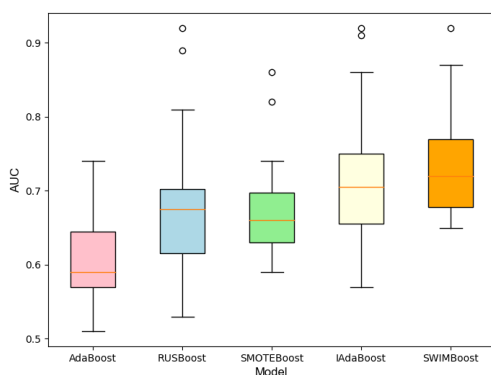


FIGURE 4. Comparison of AUC measures of predictive models such as AdaBoost, RUSBoost, SMOTEBoost, IAdaBoost and SWIMBoost.

more stable. The red lines in each box represent the median predicted results of the predictive model on 16 datasets. A higher red line represents the better performance of the predictive model. By comparing the models in the two graphs, it can be seen that the performance of the AdaBoost algorithm is distinctly more unstable and lower than that of the other four algorithms. RUSBoost and SMOTEBoost algorithms have similar performance and higher stability than AdaBoost. Moreover, the performance of IAdaBoost and SWIMBoost algorithms on G-mean and AUC measures are much higher than other three algorithms and the performance stability is not reduced.

Additionally, in order to prove the significance of IAdaBoost and SWIMBoost performance, statistical test of G-mean and AUC results were performed using Wilcoxon signed-rank test, which is a simple, safe, and robust non-parametric test for statistical comparisons of classifiers [48].

Table 7 collects the results of the Wilcoxon signed-rank test between the proposed algorithms in this article and other advanced algorithms. As the p-values show, SWIMBoost and IAdaBoost produce a statistically significant improvement in

the G-mean and AUC results. From these results we can conclude that SWIMBoost and IAdaBoost perform better than other classic imbalanced learning algorithms when dealing with the imbalanced data sets in software defect predication.

D. INFLUENCES OF VARIOUS FACTORS ON THE PERFORMANCE OF IADABOOST AND SWIMBOOST ALGORITHMS.

To investigate the effects of the number of weak classifiers on the IAdaBoost or SWIMBoost performance, we present the changing tendencies of the mean value of AUC or G-mean across 16 data sets along with the increase of the number of weak classifiers in Fig. 5. As the number of weak classifiers increases, the performance of the IAdaBoost and SWIMBoost algorithms initially improves and then decreases. This is because there is a problem of underfitting when the number of weak classifiers is too small, and there is a risk of overfitting when the number of weak classifiers is too large. IAdaBoost achieves the best performance when the number of weak classifiers is equal to 20, and SWIMBoost achieves the best performance when the number of classifiers equals 90.

We should also consider the impacts of different weak classifiers on IAdaBoost or SWIMBoost performance. When the number of weak classifiers is equal to 50, the mean value of AUC or G-mean across 16 data sets are summarized in Table 8. According to Table 8, three different weak classifiers are used in the experiment, which are decision trees (CART), support vector machine (SVM) and naive Bayes (NB). Among the three weak classifiers, CART has the best performance, and NB has the worst performance. The performance difference of weak classifiers is fluctuated at about 2%, so different weak classifiers have little impact on the performance of IAdaBoost and SWIMBoost algorithms. In general, the performance of IAdaBoost and SWIMBoost algorithms is relatively stable.

Finally, in order to investigate the relationship between classifier performance and the number of synthesized minority instances, we select six representative datasets from NASA and AEEEM repository and map out some line graphs shown in Fig. 6 to reflect performance changes of SWIMBoost classifier on these datasets. In Fig. 6, the blue line represents the AUC value and the orange line represents the G-mean value. The horizontal axis represents that the current number of minority instances is several times greater than the number of minority instances in the original dataset. For example, if $flip = 2$ means that the current number of minority instances is two times greater than the number of minority instances in the original dataset. When the horizontal axis reaches the maximum value, it means that majority and minority have the same number of instances, that is, the training set is absolutely balanced.

In Fig. 6, on most datasets, with the increase of the number of minority instances, the performance of the classifier initially is improved. The classification performance reaches the maximum value when the number of minority instances

TABLE 7. Wilcoxon signed-rank test results for the comparison of G-mean and AUC measures of all predictive models ($\alpha=0.05$, where α is significant level).

Methods	Evaluation measurement	Hypothesis	P-value	Result
SWIMBoost vs. AdaBoost	G-mean	H_0 : SWIMBoost performs better than AdaBoost.	0.0002	Accept
SWIMBoost vs. RUSBoost	G-mean	H_0 : SWIMBoost performs better than RUSBoost.	0.0065	Accept
SWIMBoost vs. SMOTEBoost	G-mean	H_0 : SWIMBoost performs better than SMOTEBoost.	0.0002	Accept
IAAdaBoost vs. AdaBoost	G-mean	H_0 : IAAdaBoost performs better than AdaBoost.	0.0011	Accept
IAAdaBoost vs. RUSBoost	G-mean	H_0 : IAAdaBoost performs better than RUSBoost.	0.0005	Accept
IAAdaBoost vs. SMOTEBoost	G-mean	H_0 : IAAdaBoost performs better than SMOTEBoost.	0.0053	Accept
SWIMBoost vs. AdaBoost	AUC	H_0 : SWIMBoost performs better than AdaBoost.	0.0002	Accept
SWIMBoost vs. RUSBoost	AUC	H_0 : SWIMBoost performs better than RUSBoost.	0.0007	Accept
SWIMBoost vs. SMOTEBoost	AUC	H_0 : SWIMBoost performs better than SMOTEBoost.	0.0002	Accept
IAAdaBoost vs. AdaBoost	AUC	H_0 : IAAdaBoost performs better than AdaBoost.	0.0019	Accept
IAAdaBoost vs. RUSBoost	AUC	H_0 : IAAdaBoost performs better than RUSBoost.	0.0003	Accept
IAAdaBoost vs. SMOTEBoost	AUC	H_0 : IAAdaBoost performs better than SMOTEBoost.	0.0084	Accept

TABLE 8. Average performances of the proposed algorithms with different weak classifiers across all data sets.

Method	Weak estimator	G-mean	AUC
IAAdaBoost	CART	0.6996	0.7038
	SVM	0.6871	0.6953
	NB	0.6866	0.7028
SWIMBoost	CART	0.7196	0.7358
	SVM	0.7066	0.7218
	NB	0.6956	0.7107

is doubled. Then the number of minority instances continue to increase, and the performance of the classifier does not increase but decreases. When the number of majority instances is equal to the number of minority instances, the performance of the classifier is not as good as that on the original dataset. Therefore, we think that the performance of classifiers is improved authentically by using oversampling methods, if we can accurately control the number of minority instances generated. The initial generated minority instances effectively expand the boundaries of minority class and increase the information of minority class, which has a positive impact for the classification. However, when the number of minority instance is more than doubled, it is inevitable that

noise will be introduced, which will have a negative impact for the classification. Another observation is that the curves of AUC and G-mean are highly similar, indicating that our measures can reasonably reflect the performance trends of the classifier.

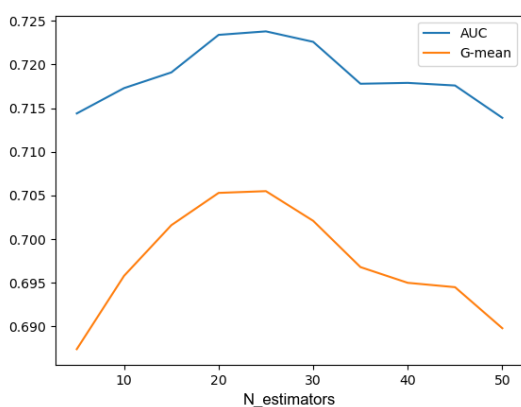
In summary, the performance of the classifier is the best when the number of minority instances is doubled by using oversampling method. In Table 5 and Table 6, the classification results of SWIMBoost on each dataset are obtained by doubling the number of minority instances.

V. THREATS TO VALIDITY

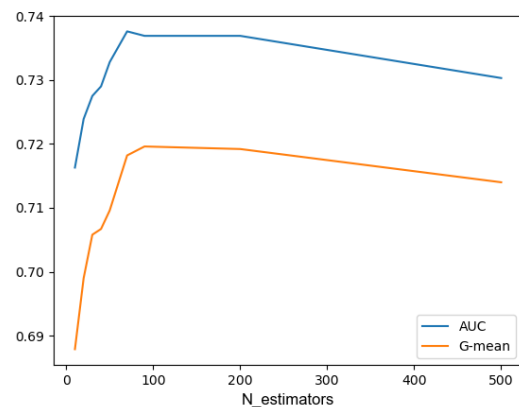
In our experiments, there are several potential threats including external validity and construct validity.

A. EXTERNAL VALIDITY

In our experiments, we chose 16 datasets from the NASA and AEEEM repositories. Although these datasets differ largely in their size and imbalance rate, they might not be representative for all software projects. In the future, we intend to reduce this threat by conducting further experiments on more defect data from open source and commercial software projects.



(a) IAAdaBoost



(b) SWIMBoost

FIGURE 5. Average performances of the proposed algorithms across all data sets (N_estimators corresponds to the maximum number of estimators).

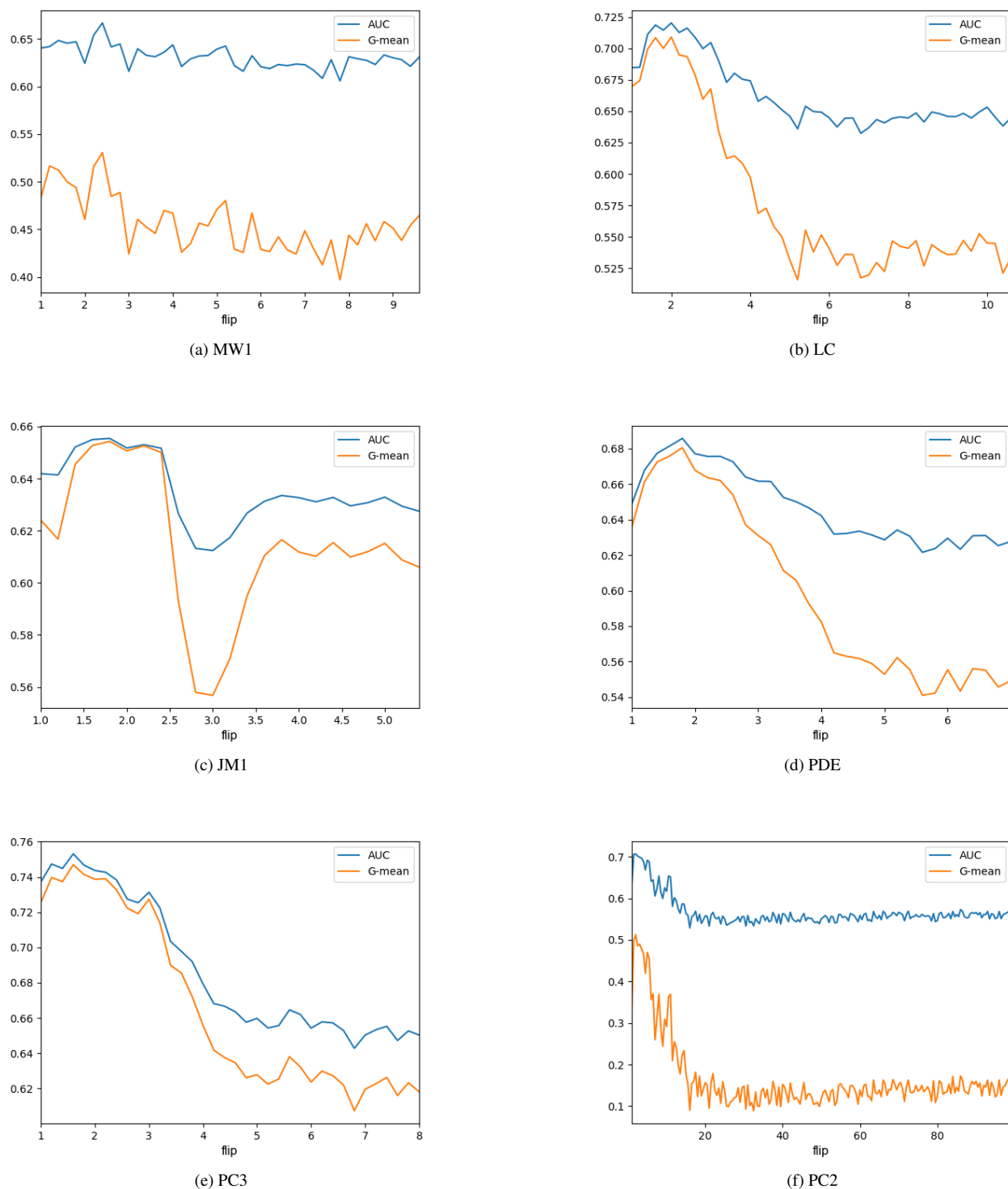


FIGURE 6. As the number of minority instances increases, the performance of the SWIMBoost algorithm changes.

B. CONSTRUCT VALIDITY

In our experiments, we select CART, SVM, and NB as our weak classifier respectively. The evaluation result may be different if the weak classifier is changed. We will experiment with other weak classifiers in our future work. Besides, Our results rely on ten times ten-fold cross validation, and this strategy has been used in the prior defect prediction studies. Whether the splitting strategy is optimal or not is uncertain. We leave the effects of different splitting strategies for future studies.

VI. CONCLUSION

This paper presents two novel methods to handle the problem of class imbalance datasets in software defect prediction. Specifically, the first one was called IAdaBoost algorithm, which introduce cost-sensitive learning into AdaBoost to make it more suitable for dealing with high imbalance rates. The other one was called SWIMBoost algorithm, which introduce oversampling methods SWIM to synthesize minority instances and thus make it perform better in issues with slight imbalance rates. Experimental results from several imbalanced data sets indicate that the proposed IAdaBoost and SWIMBoost algorithm can result in better prediction than AdaBoost, RUSBoost, and SMOTEBoost. Data sets used in our experiments contained different degrees of imbalance and different sizes, thus providing a diverse test bed. In addition, we further discuss the relationship between SWIMBoost performance and the number of synthetic minority instances and how the numbers of selected weak classifiers impact the performance of the proposed algorithm as well as the different weak classifiers.

In the course of our experiment, we find that some noises are inevitably introduced into the minority instances that generated by the over-sampling technology. Specifically, these newly generated noises will affect the performance of the classifier. Therefore, it is very meaningful to work out an algorithm to deal with noises. In addition, the algorithm proposed in this paper is specialized in dealing with unbalanced binary classification data sets. In software engineering, there are many problems with more than two classes, we will focus on the application of IAdaBoost and SWIMBoost in multi-classification. Finally, cross-project defect prediction is also an important branch in software defect prediction. It uses dataset from other projects to predict defects in projects that lack historical or sufficient local dataset, and also faces class imbalance. How the proposed methods in this paper are applied in cross project scenario is also the focus of future work.

REFERENCES

- [1] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic literature review on fault prediction performance in software engineering," *IEEE Trans. Softw. Eng.*, vol. 38, no. 6, pp. 1276–1304, Nov./Dec. 2012.
- [2] R. Malhotra, "A systematic review of machine learning techniques for software fault prediction," *Appl. Soft Comput.*, vol. 27, pp. 504518, 2015.
- [3] A. Mockus and D. M. Weiss, "Predicting risk of software changes," *Bell Labs Technical Journal*, vol. 5, no. 2, pp. 169–180, 2000.
- [4] Y. Kamei, T. Fukushima, S. McIntosh, K. Yamashita, N. Ubayashi, and A. E. Hassan, "Studying just-in-time defect prediction using crossproject models," *Empirical Software Engineering (EMSE)*, vol. 21, no. 5, pp. 2072–2106, 2015.
- [5] N. Nan and D. E. Harter, "Impact of budget and schedule pressure on software development cycle time and effort," *IEEE Transactions on Software Engineering (TSE)*, vol. 35, no. 5, pp. 624–637, 2009.
- [6] Liang, H., et al., SemL: A Semantic LSTM Model for Software Defect Prediction. *IEEE Access*, 2019. 7(): p. 83812–83824.
- [7] N. Cristianini and J. Shawe-Taylor, *An Introduction to Support Vector Machines and Other Kernel-Based Learning Methods*. New York, NY, USA: Cambridge Univ. Press, 2000.
- [8] J. R. Quinlan, "Induction of decision trees," *Mach. Learn.*, vol. 1, no. 1, pp. 81–106, 1986.
- [9] Liu, G., et al., Convolutional Neural Networks-Based Locating Relevant Buggy Code Files for Bug Reports Affected by Data Imbalance. *IEEE Access*, 2019. 7(): p. 131304–131316.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, no. 1, pp. 321–357, 2002.
- [11] A. Estabrooks, T. H. Jo, and N. Japkowicz, "A multiple resampling method for learning from imbalanced data sets," *Comput. Intell.*, vol. 20, no. 1, pp. 18–36, 2004.
- [12] H. He and E. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [13] Tao, X., et al., Self-adaptive cost weights-based support vector machine cost-sensitive ensemble for imbalanced data classification. *Information Sciences*, 2019. 487: p. 31–56.
- [14] Li, Z., et al., Heterogeneous defect prediction with two-stage ensemble learning. *Automated Software Engineering*, 2019. 26(3): p. 599–651.
- [15] S. Sharma, C. Bellinger, B. Krawczyk, O. Zaiane and N. Japkowicz, "Synthetic Oversampling with the Majority Class: A New Perspective on Handling Extreme Imbalance," *2018 IEEE International Conference on Data Mining (ICDM)*, Singapore, 2018, pp. 447–456.
- [16] D. Gray, D. Bowes, N. Davey, Y. Sun, and B. Christianson, "The misuse of the NASA metrics data program data sets for automated software defect prediction," *Proc. 15th Annu. Conf. Eval. Assessment Softw. Eng.*, 2011, pp. 96–103.
- [17] K. E. Bennin, J. Keung, A. Monden, Y. Kamei, and N. Ubayashi, "Investigating the effects of balanced training and testing datasets on effort-aware fault prediction models," in *Proc. IEEE 40th Annu. Comput. Softw. Appl. Conf.*, 2016, pp. 154–163.
- [18] G. M. Weiss and F. Provost, "The effect of class distribution on classifier learning: An empirical study," *Rutgers Univ.*, 2001.
- [19] K. Yoon and S. Kwek, "A data reduction approach for resolving the imbalanced data issue in functional genomics," *Neural Comput. Appl.*, vol. 16, no. 3, pp. 295–306, 2007.
- [20] E. Arisholm, L. C. Briand, and E. B. Johannessen, "A systematic and comprehensive investigation of methods to build and evaluate fault prediction models," *J. Syst. Softw.*, vol. 83, no. 1, pp. 2–17, 2010.
- [21] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Trans. Syst. Man Cybern. Part C (Appl. Rev.)*, vol. 42, no. 4, pp. 463–484, Jul. 2012.
- [22] P. Branco, L. Torgo, and R. P. Ribeiro, "A survey of predictive modeling on imbalanced domains," *ACM Comput. Surv.*, vol. 49, no. 2, 2016, Art. no. 31.
- [23] J. Laurikkala, "Improving Identification of Difficult Small Classes by Balancing Class Distribution," *Proc. Conf. AI in Medicine in Europe: Artificial Intelligence Medicine*, pp. 63–66, 2001.
- [24] T. Jo and N. Japkowicz, "Class Imbalances versus Small Disjuncts," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 40–49, 2004.
- [25] C. Elkan, "The Foundations of Cost-Sensitive Learning," *Proc. Int'l Joint Conf. Artificial Intelligence*, pp. 973–978, 2001.
- [26] M.A.Maloof, "Learning When Data Sets Are Imbalanced and When Costs Are Unequal and Unknown," *Proc. Int'l Conf. Machine Learning, Workshop Learning from Imbalanced Data Sets II*, 2003.
- [27] K. McCarthy, B. Zabar, and G.M. Weiss, "Does Cost-Sensitive Learning Beat Sampling for Classifying Rare Classes?" *Proc. Int'l Workshop Utility-Based Data Mining*, pp. 69–77, 2005.
- [28] Z. Qin, A. T. Wang, C. Zhang, and S. Zhang, "Cost-sensitive classification with k-nearest neighbors," in *Proc. Int. Conf. Knowl. Sci. Eng. Manage.*, 2013, pp. 112–131.

- [29] Y. Sahin, S. Bulkan, and E. Duman, "A cost-sensitive decision tree approach for fraud detection," *Expert Syst. Appl.*, vol. 40, no. 15, pp. 5916–5923, 2013.
- [30] X.Y. Liu and Z.H. Zhou, "Training Cost-Sensitive Neural Networks with Methods Addressing the Class Imbalance Problem," *IEEE Trans. Knowledge and Data Eng.*, vol. 18, no. 1, pp. 63–77, Jan. 2006.
- [31] J. Xu, Y. Cao, H. Li, and Y. Huang, "Cost-sensitive learning of SVM for ranking," in *Proc. Eur. Conf. Mach. Learning*, 2006, pp. 833–840.
- [32] J. Kittler, M. Hatef, R. Duin, and J. Matas, "On combining classifiers," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 3, pp. 226–239, Mar. 1998.
- [33] L. Breiman, "Bagging predictors," *Mach. Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [34] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [35] Freund, Y. and R.E. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting. 1995: Barcelona, Spain. p. 23 - 37.
- [36] W. Fan, S. J. Stolfo, J. Zhang, and P. K. Chan, "Adacost: Misclassification cost-sensitive boosting," presented at the 6th Int. Conf. Mach. Learning, pp. 97–105, San Francisco, CA, 1999.
- [37] K. M. Ting, "A comparative study of cost-sensitive boosting algorithms," in *Proc. 17th Int. Conf. Mach. Learning, Stanford, CA, 2000*, pp. 983–990.
- [38] Y. Sun, M. S. Kamel, A. K. Wong, and Y. Wang, "Cost-sensitive boosting for classification of imbalanced data," *Pattern Recog.*, vol. 40, no. 12, pp. 3358–3378, 2007.
- [39] Li, K., et al., Improved PSO_AdaBoost ensemble algorithm for imbalanced data. *Sensors (Switzerland)*, 2019. 19(6).
- [40] N. V. Chawla, A. Lazarevic, L. O. Hall, and K. W. Bowyer, "SMOTE-Boost: Improving prediction of the minority class in boosting," in *Proc. Knowl. Discov. Databases*, 2003, pp. 107–119.
- [41] C. Seiffert, T. Khoshgoftaar, J. Van Hulse, and A. Napolitano, "Rusboost: A hybrid approach to alleviating class imbalance," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 40, no. 1, pp. 185–197, Jan. 2010.
- [42] Deng, X., et al., Dynamic clustering method for imbalanced learning based on AdaBoost. *Journal of Supercomputing*, 2020. 76(12): p. 9716 - 9738.
- [43] X.-Y. Liu, J. Wu, and Z.-H. Zhou, "Exploratory undersampling for class-imbalance learning," *IEEE Trans. Syst., Man, Cybern. B, Appl. Rev.*, vol. 39, no. 2, pp. 539–550, 2009.
- [44] M. D'Ambros, M. Lanza, and R. Robbes, "Evaluating defect prediction approaches: A benchmark and an extensive comparison," *Empirical Softw. Eng.*, vol. 17, no. 4–5, pp. 531–577, 2012.
- [45] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the NASA software defect datasets," *IEEE Trans. Softw. Eng.*, vol. 39, no. 9, pp. 1208–1215, Sep. 2013.
- [46] Zhang, J., et al., CDS: A Cross-Version Software Defect Prediction Model With Data Selection. *IEEE Access*, 2020. 8(): p. 110059-110072.
- [47] Qiu, Y., et al., Automatic Feature Exploration and an Application in Defect Prediction. *IEEE Access*, 2019. 7(): p. 112097-112112.
- [48] Demsar, J., Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 2006. 7: p. 1 - 30.



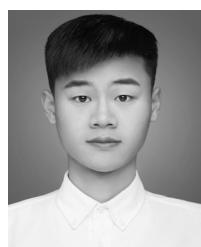
XINGQI WANG received his Bachelor and Master degree from Harbin Institute of Technology in 1997 and 1999, respectively, and PhD degree from Zhejiang University in 2002. He is a professor in School of Computer Science, Hangzhou Dianzi University, China. As a researcher, he visited CERCIA, University of Birmingham, UK from 2005 to 2006. His research interests include machine learning, data mining, and multimedia content analysis.



DAN WEI received her PhD degree from Xiamen University in 2013. She is a lecturer in School of Computer Science at Hangzhou Dianzi University, China. Her research interests mainly include machine learning, data mining, and Bioinformatics.



BIN CHEN received his Ph.D degree in Tokyo University of Agriculture and Technology, Japan, in 2013. He is a lecture at the School of Computer Science, Hangzhou Dianzi University, China. His research interests include software engineering, machine Learning, and handwriting character recognition.



JIANMING ZHENG received the B.E. degree from Shandong Agricultural University, Shandong, China, in 2019, where he is currently pursuing the M.E. degree in Computer science and technology from Hangzhou Dianzi University, Hangzhou, China, with a focus on software defect prediction and imbalanced learning.



YANLI SHAO received her Ph.D degree in Zhejiang University, China, in 2016. She used to work at the Institute of Automation, Chinese Academy of Sciences from 2017 to 2018. Currently, she is an associate researcher at the School of Computer Science, Hangzhou Dianzi University, China. Her research interests include Intelligent software engineering, CAD/CAE, MBSE, and artificial intelligence.

...